FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	DELA CRUZ, ROBIE SANTOS, TAYSIAN
Section:	AN43
Professor:	MS. TAN
Date:	3/11/2026

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Our pair divided the work by having one member organize the disaster cases and similarity scoring while the other developed the adaptation rules and console output. We reviewed the program together to check whether the retrieved case fit the new event and whether the recommended changes followed the input conditions. One challenge was deciding how to compare categories such as hazard type with numerical values such as severity and affected population. We addressed this by assigning weights to each attribute and normalizing the numerical comparisons. We also had to make sure the response changed when the event was more severe, affected more people, or had limited resources. This activity helped us understand how a CBR program reuses prior experience and retains new cases for later decisions.

Case Studies:

Case Study 1: Disaster Response Decision Support System
A CBR-based DSS helps local government units respond to natural disasters. • Past cases include typhoons, floods, and earthquakes. • New event: *Typhoon with expected flooding in urban areas.* • System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*. • It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.

- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.

Student submits a math solution with repeated errors.

System retrieves past students with similar error patterns (e.g., misusing formulas).

It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

```
"""Case-Based Reasoning demo for a Disaster Response Decision Support System.

Run: python TA3_CBR_Disaster_Response.py
The program saves retained cases to case_base.json beside this script.
"""

import json
from pathlib import Path

CASE_FILE = Path(__file__).with_name("case_base.json")

# Seed case base: prior disaster situations and their successful response plans.
INITIAL_CASES = [
    {
        "case_id": "D001",
        "hazard": "typhoon",
        "severity": 5,
```

```
    "people_affected": 1200,
    "resources": "adequate",
     "solution": "Open evacuation centers, deploy emergency teams, and distribute food, water, and medical supplies.",
  },
  {
    "case_id": "D002",
    "hazard": "flood",
    "severity": 4,
    "people_affected": 800,
    "resources": "limited",
     "solution": "Evacuate households in low-lying areas, prioritize older adults and children, and request boats and relief goods.",
  },
  {
    "case_id": "D003",
    "hazard": "earthquake",
    "severity": 5,
    "people_affected": 950,
    "resources": "moderate",
     "solution": "Set up safe assembly areas, assess damaged buildings, and coordinate search-and-rescue and medical teams.",
  },
  {
    "case_id": "D004",
    "hazard": "typhoon",
    "severity": 3,
    "people_affected": 350,
    "resources": "moderate",
    "solution": "Issue local warnings, prepare a nearby shelter, and pre-position basic food, water, and first-aid supplies.",
  },
  {
    "case_id": "D005",
    "hazard": "flood",
    "severity": 2,
    "people_affected": 120,
    "resources": "adequate",
     "solution": "Monitor water levels, clear drainage routes, and prepare transport for residents who may need evacuation.",
  },
]

# Weighted attributes sum to 1.00; population similarity is scaled to 1,500 people.
```

```
WEIGHTS = {"hazard": 0.35, "severity": 0.25, "people_affected": 0.25, "resources": 0.15}
RESOURCE_LEVEL = {"limited": 1, "moderate": 2, "adequate": 3}

def load_case_base():
    """Load retained experience, or start with the five sample cases."""
    if CASE_FILE.exists():
        try:
            with CASE_FILE.open("r", encoding="utf-8") as file:
                saved_cases = json.load(file)
            if isinstance(saved_cases, list) and saved_cases:
                return saved_cases
        except (OSError, json.JSONDecodeError):
            print("Saved case file could not be read; using the built-in cases.")
    return [case.copy() for case in INITIAL_CASES]

def save_case_base(cases):
    """Persist the case base as readable JSON for the next program run."""
    with CASE_FILE.open("w", encoding="utf-8") as file:
        json.dump(cases, file, indent=2, ensure_ascii=False)

def attribute_similarity(new_case, old_case):
    """Return a weighted similarity from 0.00 (different) to 1.00 (same)."""
    hazard_match = 1.0 if new_case["hazard"] == old_case["hazard"] else 0.0
    severity_match = 1 - abs(new_case["severity"] - old_case["severity"]) / 4
    population_match = max(
        0.0,
        1 - abs(new_case["people_affected"] - old_case["people_affected"]) / 1500,
    )
    resource_difference = abs(
        RESOURCE_LEVEL[new_case["resources"]] - RESOURCE_LEVEL[old_case["resources"]]
    )
    resource_match = 1 - resource_difference / 2

    return (
        WEIGHTS["hazard"] * hazard_match
        + WEIGHTS["severity"] * severity_match
        + WEIGHTS["people_affected"] * population_match
        + WEIGHTS["resources"] * resource_match
    )

def retrieve_cases(new_case, cases):
    """Rank past cases from most to least similar and return the ranking."""
    return sorted(
        ((attribute_similarity(new_case, case), case) for case in cases),
```

```
        key=lambda result: result[0],
        reverse=True,
    )

def adapt_solution(new_case, retrieved_case):
    """Reuse the retrieved plan, then adapt priorities to the new conditions."""
    actions = [retrieved_case["solution"]]

    if new_case["severity"] >= 4 and new_case["severity"] > retrieved_case["severity"]:
        actions.append("Escalate to the city or municipal emergency operations center and issue urgent public alerts.")
    elif new_case["severity"] <= 2:
        actions.append("Use a targeted response and continue monitoring before expanding the evacuation area.")

    if new_case["people_affected"] > retrieved_case["people_affected"]:
        actions.append("Increase shelter capacity, transport, and relief supplies to cover the larger population.")
    elif new_case["people_affected"] < retrieved_case["people_affected"]:
        actions.append("Scale the deployment to the smaller affected population while keeping reserve supplies ready.")

    if new_case["resources"] == "limited":
        actions.append("Prioritize people at greatest risk and request additional responders and supplies from nearby LGUs.")
    elif new_case["resources"] == "adequate" and retrieved_case["resources"] != "adequate":
        actions.append("Use available staff and supplies to open additional support points where needed.")

    return actions

def ask_choice(prompt, choices):
    """Prompt until the user enters one of the allowed choices."""
    choices_text = ", ".join(choices)
    while True:
        answer = input(f"{prompt} ({choices_text}): ").strip().lower()
        if answer in choices:
            return answer
        print(f"Please enter one of these values: {choices_text}.")

def ask_integer(prompt, minimum):
    """Prompt until a valid integer at or above the minimum is entered."""
    while True:
        try:
```

```
            value = int(input(prompt).strip())
            if value >= minimum:
                return value
        except ValueError:
            pass
        print(f"Enter a whole number of at least {minimum}.")

def main():
    cases = load_case_base()
    print("=" * 68)
    print("DISASTER RESPONSE CBR DECISION SUPPORT SYSTEM")
    print("=" * 68)
    print("Describe the new event. Similarity ranking uses the stored case attributes.\n")

    new_case = {
        "hazard": ask_choice("Hazard", ["typhoon", "flood", "earthquake"]),
        "severity": ask_integer("Severity (1 = low, 5 = critical): ", 1),
        "people_affected": ask_integer("Estimated people affected: ", 0),
        "resources": ask_choice("Available resources", ["limited", "moderate", "adequate"]),
    }
    if new_case["severity"] > 5:
        print("Severity is capped at 5 (critical).")
        new_case["severity"] = 5

    ranking = retrieve_cases(new_case, cases)
    best_score, best_case = ranking[0]
    adapted_actions = adapt_solution(new_case, best_case)

    print("\n--- New problem ---")
    print(
        f"{new_case['hazard'].title()} | severity {new_case['severity']}/5 | "
        f"{new_case['people_affected']:,} people affected | resources: {new_case['resources']}"
    )
    print("\n--- Similarity ranking ---")
    for score, case in ranking:
        print(f"{case['case_id']} ({case['hazard']}, severity {case['severity']}): {score:.3f}")
            print(f"\nRetrieved case: {best_case['case_id']} ({best_case['hazard']}) — similarity
{best_score:.3f}")
    print("\n--- Adapted response plan ---")
    for number, action in enumerate(adapted_actions, start=1):
        print(f"{number}. {action}")

    retain = ask_choice("Retain this solved case for future use?", ["yes", "no"])
    if retain == "yes":
```

```
        new_id = f"D{len(cases) + 1:03d}"
        new_case["case_id"] = new_id
        new_case["solution"] = " ".join(adapted_actions)
        cases.append(new_case)
        save_case_base(cases)
        print(f"\nNew case saved as {new_id} in {CASE_FILE.name}.")
    else:
        print("\nCase was not retained.")

    print(f"\n--- Updated case base ({len(cases)} cases) ---")
    for case in cases:
        print(
            f"{case['case_id']}: {case['hazard']}, severity {case['severity']}, "
            f"{case['people_affected']:,} affected, resources {case['resources']}"
        )

if __name__ == "__main__":
    main()
```